

Data Structures & Algorithms พื้นฐานสำหรับผู้เริ่มต้น

บทสรุปย่อ: Array, Linked List, Stack, Queue, Tree & Big-O Notation ฉบับเข้าใจง่าย

บทที่ 1: การวิเคราะห์ความซับซ้อนของอัลกอริทึม (Asymptotic Analysis)

1.1 ความสำคัญของ Big-O Notation

ในการเขียนโปรแกรม เราไม่ได้วัดประสิทธิภาพด้วยเวลาจริงเป็นวินาที เพราะขึ้นอยู่กับสเปกฮาร์ดแวร์ แต่เราวัดด้วย "จำนวนรอบการทำงานตามขนาด Input n "

นิยามทางคณิตศาสตร์: $f(n) = O(g(n))$ เมื่อมีค่าคงที่ $c > 0$ และ n_0 ที่ทำให้ $f(n) \leq c \cdot g(n)$ สำหรับทุก $n \geq n_0$

1.2 ลำดับความเร็วของ Big-O จากดีที่สุดไปแย่ที่สุด

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

บทที่ 2: ต้นไม้ค้นหาแบบทวิภาค (Binary Search Tree - BST)

2.1 คุณสมบัติของ BST

BST คือโครงสร้างข้อมูลแบบ Tree ที่ในแต่ละ Node จะมีคุณสมบัติ:

- ข้อมูลใน Node ลูกทางซ้ายทั้งหมด จะมีค่าน้อยกว่า Node ปัจจุบัน
- ข้อมูลใน Node ลูกทางขวาทั้งหมด จะมีค่ามากกว่า Node ปัจจุบัน

Time Complexity ของ BST:

กรณีทั่วไป (Balanced): การค้นหา แทรก ลบ ใช้เวลา $O(\log n)$

กรณีแย่ที่สุด (Degenerate กลายเป็นเส้นตรง): ใช้เวลา $O(n)$ จึงจำเป็นต้องมี Self-Balancing Trees เช่น AVL Tree หรือ Red-Black Tree